Assignment1: Prepare a test strategy document based on the domain, using a chain-of-thought approach

Prompt 1: As a Senior Test Lead, generate a test strategy document for the Telecom fulfilment services where customer places the order online and the testing covers the entire end to end journey till the order is provisioned and user is able to access the service. There are multiple products involved like Broadband, Mobile, VOIP. Important : Ask me 7 questions one after the other for test strategy document generation. Provide the next set of questions after i respond to previous question. Strategy document should follow industry standards and should include functional testing, automation testing and performance testing. It must outline resources, project timelines, testing approach,

Great — I'll help you build a solid, industry-grade test strategy step by step.

I'll ask **7 questions one at a time**, and based on your answers, I'll progressively shape the document.

Question 1 of 7: Scope & Systems Involved

To define the test strategy properly, I need to understand the **system landscape**.

☞ **What are the key systems involved in the end-to-end telecom fulfilment journey?**

For example (you can confirm/add/remove):

- Customer-facing channels (Web / Mobile App)
- Order Management System (OMS)
- CRM
- Billing System
- Provisioning Systems (Broadband / Mobile / VOIP)
- Network Systems (e.g., activation platforms)
- Third-party integrations (KYC, payment gateways, etc.)

💡 Please list:

1. All systems involved
2. Any known integration points between them
3. Whether this is a **greenfield, enhancement, or migration project**

Once you answer this, I'll move to **Question 2**.

Qn1 Ans: Include All systems mentioned except third party payment gateways and KYC. This is a greenfield project for a new customer

Great, that gives a clear starting point — greenfield programs are ideal for setting strong testing foundations 👍

Question 2 of 7: Order Journey & Product Variations

To design end-to-end coverage, I need to understand how the **customer journey and product flows differ**.

☞ **Can you describe the high-level order journey and key variations for each product?**

Please include:

1. **Typical Order Flow (common steps)**
   Example:
   - Product selection
   - Plan configuration
   - Address validation
   - Order submission
   - Order processing
   - Provisioning
   - Activation
2. **Product-specific differences**
   - How does **Broadband** fulfilment differ from **Mobile** and **VOIP**?
   - Any dependencies like:
     - Physical installation (Broadband)
     - SIM activation (Mobile)
     - Number assignment (VOIP)
3. **Any special scenarios**
   - New customer vs existing customer
   - Port-in / number portability
   - Upgrades / add-ons

Once you share this, I'll move to **Question 3**.

Qn 2 Ans : 1) Order flow is correct. No additional steps. 2) each product will have separate plans and there are few bundle products as well where the customer can select different products as a combo pack. Consider the dependencies mentioned. 3) For existing broadband customer, there are plan upgrade scenarios, discontinuation of the service, Service on hold due to non-payment.

Perfect — that gives a solid view of the journey and key variations, especially with bundles and lifecycle scenarios.

Question 3 of 7: Environments & Test Data Strategy

A strong test strategy depends heavily on **environment setup and data management**, especially for telecom systems with multiple integrations.

☞ **Please provide details on the test environments and data strategy:**

1. **Environments Available**
    - How many environments are planned? (e.g., SIT, UAT, Pre-Prod)
    - Will there be a **dedicated integration testing environment**?
    - Any environment shared with other teams?
2. **Test Data Management**
    - Will test data be:
        - Synthetic (mocked)
        - Production-like (masked data)
    - How will you handle:
        - Customer data
        - Phone numbers / SIM data
        - Address data for broadband availability
3. **Provisioning Simulation**
    - Are real provisioning systems used in lower environments, or:
        - Simulators / stubs for network activation?

Once you answer this, I'll move to **Question 4**.

Qn3 Ans: Environments : Will have 1 dev environment, 2 test environments, 1 UAT and 1 pre-prod Test Data Management: Will be using Synthetic data and all sensitive data will be masked and stored into DB. For phone numbers and addresses, third party API will be called to fetch and use the relevant data. For test and UAT environments we do have Stubs which will handle the activation

Excellent — this is shaping up well. Your use of stubs and synthetic data is especially important for a greenfield telecom stack.

Question 4 of 7: Testing Scope & Types

Now let's define **what all testing types need to be included** in the strategy.

☞ **Which testing types do you want to include in scope?**

Please confirm and/or expand on the below:

Core Testing Types

- Functional Testing (UI + API)

- System Integration Testing (SIT)
- End-to-End Testing (E2E across all systems)

Advanced Testing

- Automation Testing (UI / API / both?)
- Performance Testing (Load, Stress, Volume)
- Security Testing (basic or full scope?)
- Data Validation / Database Testing

Special Telecom Scenarios

- Provisioning validation (stub vs real behavior)
- Order fallout / failure handling
- Retry & recovery scenarios
- Batch job testing (if any)

Out of Scope (if any)

- Anything you explicitly want excluded?

💡 Also mention:

- Any **compliance or regulatory testing** needed (if applicable)

Once you respond, I'll move to **Question 5**.

Qn4 Ans: Security testing is handled by separate team. So do not focus on that. Rest all is fine. Batch testing should be included. Adhere to GDPR and EECC standards

Great — that clarifies scope nicely, especially with compliance and batch testing included.

Question 5 of 7: Automation Strategy & Tooling

Now let's define how automation will be approached, which is critical for a greenfield program.

☞ **Please provide details on automation expectations:**

1. **Automation Scope**
    - Do you want automation for:
        - UI (Web / Mobile)?
        - API layer?

        - End-to-End journeys?
    - Any priority areas? (e.g., critical customer journeys, provisioning flows)
2. **Tooling Preferences (if any)**
    - UI Automation: (e.g., Selenium, Cypress, Playwright)
    - API Automation: (e.g., RestAssured, Postman/Newman)
    - Performance Testing: (e.g., JMeter, Gatling, LoadRunner)
3. **CI/CD Integration**
    - Will automation be integrated into CI/CD pipelines? (e.g., Jenkins, GitHub Actions, Azure DevOps)
    - Expected frequency:
        - On every build?
        - Nightly regression?
        - Pre-release?
4. **Automation Goals**
    - Target automation coverage % (if any)
    - Key objective:
        - Regression reduction
        - Faster releases
        - Early defect detection

Once you answer this, I'll move to **Question 6**.

Qn5 Ans: UI and API Automation is in scope. UI for End to End journeys using playwright and API Automation using POSTMAN. For performance is it Jmeter. CI/CD integration will happen on every build during functional testing and during regression it will be nightly. target Test Automation coverage is 75% with key focus on early defect deduction and manual efforts reduction during regression.

Great — very clear and aligned with modern automation practices 👍

Question 6 of 7: Team Structure, Roles & Responsibilities

To complete the strategy, I need to define the **resourcing model and responsibilities**.

☞ **Please provide details on the testing team structure:**

1. **Team Size & Composition**
    - Number of:
        - Test Leads
        - Functional Testers
        - Automation Engineers
        - Performance Testers
    - Any shared resources across projects?
2. **Skill Distribution**
    - Do testers handle both **functional + automation**, or is it specialized?
    - Any dedicated **test data / environment support roles**?

3. **Onshore / Offshore Model**
    - Is the team:
        - Fully offshore
        - Onshore + offshore (hybrid)
    - Any timezone dependencies?
4. **Stakeholder Interaction**
    - Which teams will QA closely work with?
        - Dev team
        - Business / Product owners
        - Network / provisioning teams

Once you answer this, I'll ask the **final Question 7**, and then generate your **complete Test Strategy Document**.

Qn6 Ans: Team comprises of 1 QA Manager, 1 Senior Test lead, 15 testers out of which 10 are for manual and 4 are for automation and 1 for performance. Depending on the project status, automation folks might help with functional manual testing on need basis. Full team is working offshore and might have client calls at UK hours. QA team work with Dev, product owner

Excellent — that gives a complete picture of the team and operating model.

Question 7 of 7: Project Timeline, Releases & Success Criteria

This is the final piece needed to shape the strategy document.

☞ **Please provide details on timelines and delivery expectations:**

1. **Project Timeline**
    - Overall duration (e.g., 6 months, 12 months)
    - Key phases (if known):
        - Design
        - Build
        - Testing cycles
        - UAT
        - Go-live
2. **Release Strategy**
    - How are releases planned?
        - Agile (sprints, e.g., 2 weeks)
        - Waterfall
        - Hybrid
    - Frequency of releases?
3. **Entry & Exit Criteria**
    - Any defined:
        - Entry criteria for testing?
        - Exit criteria (e.g., defect thresholds, pass %)?

4. **Success Metrics (KPIs)**
    - What defines success for QA?
        - Defect leakage %
        - Automation coverage
        - Test execution progress
        - SLA adherence

Once you share this, I'll generate a **complete, industry-standard Test Strategy Document** tailored to your telecom fulfilment program (including functional, automation, performance, timelines, resources, and governance).

Qn7 Ans: Project duration is of 9 months and planned to incorporate Agile model with 3 weeks sprint. Standard STLC is to be followed. industry standards KPIs to be met and nothing additional.

Perfect — I now have everything needed to build a comprehensive, industry-standard **Test Strategy Document** tailored to your telecom fulfilment program.

Test Strategy Document

Telecom Fulfilment Services (Broadband, Mobile, VOIP)

---

1. Introduction

This document outlines the **test strategy** for the end-to-end telecom fulfilment system covering customer order placement through to service provisioning and activation.

The scope includes multiple product lines:

- Broadband
- Mobile
- VOIP
- Bundle/Combo offerings

The strategy is designed for a **greenfield implementation**, ensuring robust quality engineering practices aligned with **industry standards**, **GDPR**, and **EECC compliance**.

---

2. Objectives

- Ensure high-quality delivery of telecom fulfilment journeys
- Validate end-to-end order lifecycle across all systems
- Achieve **75% automation coverage**
- Enable early defect detection via CI/CD

- Minimize production defects and order fallouts
- Ensure performance and scalability under expected loads

---

3. Scope

In Scope

- End-to-end order journey:
  - Product selection → Order submission → Fulfilment → Provisioning → Activation
- Products:
  - Broadband (with installation dependencies)
  - Mobile (SIM activation)
  - VOIP (number provisioning)
  - Bundle products
- Customer scenarios:
  - New customer onboarding
  - Existing customer plan upgrades
  - Service discontinuation
  - Service suspension (non-payment)
- Batch job processing
- Order fallout and recovery scenarios
- Stub-based provisioning validation

Out of Scope

- Security testing (handled by a separate team)
- Third-party payment/KYC systems

---

4. System Landscape

The testing will cover the following integrated systems:

- Customer Web Channel
- Order Management System (OMS)
- CRM System
- Billing System
- Provisioning Systems (Broadband, Mobile, VOIP)
- Network Activation Systems

All systems are tightly integrated and will be validated through **end-to-end flows**.

---

5. Test Approach

5.1 Functional Testing

- UI and API validation
- Product configuration and order flows
- Validation of:
    - Plans and bundles
    - Customer lifecycle scenarios
    - Order status transitions
- Negative and edge case testing
- Data validation across systems

5.2 Integration Testing (SIT)

- Validate system interactions:
    - OMS ↔ CRM
    - OMS ↔ Billing
    - OMS ↔ Provisioning
- Stub-based testing for provisioning systems
- Data consistency validation

5.3 End-to-End Testing

- Full journey validation:
    - Order capture → Fulfilment → Activation
- Cross-product scenarios
- Multi-system orchestration validation
- Order fallout and retry handling

5.4 Automation Testing

Tools:

- UI: Playwright
- API: Postman (Newman)

Approach:

- Automate critical customer journeys:
    - Order placement
    - Provisioning flows
    - Plan upgrades
- API-first automation for stability
- CI/CD integration:
    - On every build (smoke tests)
    - Nightly regression runs

Target:

- 75% regression coverage

Benefits:

- Faster feedback
- Reduced manual regression effort
- Early defect detection

5.5 Performance Testing

Tool: JMeter

Scope:

- Load testing (normal peak load)
- Stress testing (beyond capacity)
- Volume testing (bulk orders, batch jobs)

Key Scenarios:

- Concurrent order placements
- High-volume provisioning requests
- Batch job execution

Metrics:

- Response time
- Throughput
- System stability

5.6 Batch Testing

- Validate scheduled jobs:
    - Billing updates
    - Order status updates
- Data reconciliation checks
- Failure and recovery validation

5.7 Compliance Testing

- GDPR:
    - Data masking

    - Secure handling of customer data
- EECC:
    - Telecom regulatory compliance

---

6. Test Environments

Environment	Purpose
Dev	Unit & initial integration testing
Test Env 1	SIT
Test Env 2	Regression / E2E
UAT	Business validation
Pre-Prod	Production simulation

Provisioning:

- Stubs used in Test & UAT environments

---

7. Test Data Strategy

- Synthetic test data generation
- Masked sensitive data stored in DB
- Third-party APIs used for:
    - Phone numbers
    - Address validation

---

8. Entry & Exit Criteria

Entry Criteria

- Requirements finalized
- Test environment ready
- Test data available
- Build deployed

Exit Criteria

- All critical test cases executed
- No open critical/high defects
- Test coverage achieved
- Regression suite passed
- Stakeholder sign-off

9. Test Lifecycle (STLC)

- Requirement Analysis
- Test Planning
- Test Design
- Test Execution
- Defect Management
- Test Closure

---

10. Defect Management

- Centralized defect tracking tool (e.g., JIRA)
- Severity & priority classification
- Daily triage calls
- Root cause analysis for critical defects

---

11. Team Structure

Role	Count
QA Manager	1
Senior Test Lead	1
Manual Testers	10
Automation Engineers	4
Performance Tester	1

Model:

- Fully offshore team
- Collaboration with:
  - Development team
  - Product owners
- UK timezone overlap for client interactions

---

12. Resource Utilization Strategy

- Automation engineers support manual testing during peak phases
- Shift-left testing approach
- Parallel execution across environments

---

13. Project Timeline

- Total Duration: **9 Months**
- Methodology: **Agile (3-week sprints)**

High-Level Phases

- Sprint 0: Setup & planning
- Sprint cycles: Development + Testing
- Final Sprints: Regression + UAT
- Pre-Prod validation
- Go-Live

---

14. Release Strategy

- Agile-based incremental releases
- Continuous integration
- Nightly regression cycles
- Pre-release full regression

---

15. Risks & Mitigation

Risk	Mitigation
Environment instability	Dedicated environment support
Stub limitations	Periodic real-system validation
Data dependency	Automated data generation
Integration failures	Early SIT cycles

---

16. Key Metrics (KPIs)

- Defect Density
- Defect Leakage
- Test Coverage (%)
- Automation Coverage (Target: 75%)
- Test Execution Progress
- Pass/Fail Rate
- SLA adherence

---

17. Governance & Reporting

- Daily stand-ups
- Weekly QA status reports

- Sprint review updates
- Defect triage meetings
- Stakeholder dashboards

---

18. Tools Summary

Area	Tool
Test Management	JIRA / Test Management Tool
UI Automation	Playwright
API Automation	Postman/Newman
Performance	JMeter

---

19. Conclusion

This strategy ensures:

- End-to-end validation across complex telecom ecosystems
- Strong automation foundation
- Scalable performance validation
- Compliance with telecom regulations

It is designed to support **high-quality releases, faster delivery cycles, and reduced production risks**.

Assignment2: Design the automation framework structure using a chain-of-thought approach

Prompt: As a Senior Automation lead, generate the Automation framework from scratch for the below project : Telecom fulfilment services where customer places the order online and the testing covers the entire end to end order journey till the order is provisioned and user is able to access the service. There are multiple products involved like Broadband, Mobile, VOIP. Important : Ask me 6 questions one after the other for automation framework generation. Provide the next set of questions after i respond to previous question. Automation framework should follow the industry standards

Great — let's design a **robust, industry-standard automation framework** tailored for your telecom fulfilment system.

I'll ask **6 focused questions one at a time**, and then generate a complete framework (architecture, folder structure, design patterns, CI/CD, reporting, etc.).

Question 1 of 6: Tech Stack & Application Details

To design the framework correctly, I need clarity on the **application under test and tech constraints**.

☞ Please provide:

1. **Application Type**
   - Web only?
   - Mobile (native / hybrid)?
   - Both?
2. **Technology Stack (if known)**
   - Frontend (e.g., React, Angular)
   - Backend (e.g., Java, Node.js, microservices?)
3. **APIs**
   - REST / GraphQL?
   - Any API gateway involved?
4. **Authentication Mechanism**
   - Login-based (username/password)?
   - OTP / MFA?
   - SSO?

💡 This will help decide:

- Framework design (UI vs API-first)
- Authentication handling strategy
- Test layering approach

Once you answer, I'll move to **Question 2**.

Ans1: Application type is web only. For front end it is react and for backend it is node.js. APIs used are REST and the application supports Login using username and password

Perfect — that's a clean and modern stack (React + Node.js + REST), which allows us to design a strong **API-first + UI layered automation framework**.

Question 2 of 6: Test Scope for Automation

Now I need to define **what exactly will be automated**, which directly impacts framework design and modularization.

☞ Please clarify:

1. **Automation Coverage Scope**
    - Do you want to automate:
        - Only critical E2E journeys?
        - Or include:
            - UI component-level tests?
            - API-level validations for all services?
2. **Priority Flows (must-have automation)**
   From your telecom domain, confirm if these should be included:
    - New order placement (Broadband / Mobile / VOIP)
    - Bundle product order
    - Order provisioning & activation validation
    - Plan upgrade (existing customer)
    - Service suspension / resume
    - Order failure / retry scenarios
3. **Layer Preference**
    - Should we follow:
        - **API-first approach (recommended)** with minimal UI
        - Or heavy **UI E2E coverage**

💡 This helps define:

- Test pyramid (UI vs API ratio)
- Reusability strategy
- Execution speed optimization

Once you respond, I'll move to **Question 3**.

Ans2: Need to automate the E2E journeys and also include API level validations for all the services. Keep all the priority flows mentioned and layer preference will be 50-50 for both API and UI

Got it — a **balanced 50-50 API + UI strategy** with full E2E coverage. That's ambitious but absolutely doable with the right framework design.

Question 3 of 6: Framework Design Preferences

Now let's define the **core architecture and coding standards** for the framework.

☞ Please provide your preferences:

1. **Programming Language**
    - JavaScript / TypeScript (recommended for Playwright + Node stack?)
    - Any specific preference?
2. **Framework Type**
    - For UI (Playwright):
        - Do you prefer **Page Object Model (POM)**?
        - Or more advanced:
            - POM + Service Layer (recommended)
            - Screenplay Pattern
3. **API Automation Structure**
    - Should API tests be:
        - Independent (pure API testing)
        - Or tightly integrated with UI flows (for E2E validation)?
4. **BDD vs Non-BDD**
    - Do you want:
        - BDD (Cucumber/Gherkin)
        - Or standard test framework (Playwright test runner / Jest)

💡 This will define:

- Code structure
- Maintainability
- Readability for business stakeholders

Once you answer, I'll move to **Question 4**.

Ans3: Use playwright tool with Typescript as the programming language. POM is a must for robustness and have API Automation integrated with UI flows. Use standard test framework

Excellent — that's a strong and scalable choice: **Playwright + TypeScript + POM + integrated API flows**. This will give you both maintainability and speed.

Question 4 of 6: Test Data, Environment & Configuration Management

A telecom E2E framework heavily depends on **dynamic data and environment control**, so this is critical.

☞ Please provide details on:

1. **Test Data Strategy for Automation**
    - Should tests:
        - Generate data dynamically (recommended)?
        - Or use predefined datasets?
    - Any need for:
        - Reusable customer profiles?
        - Product/catalog-driven data?
2. **Environment Handling**
    - Will tests run across multiple environments (e.g., SIT, UAT, Pre-Prod)?
    - Should environment switching be:
        - Config-based (recommended)
        - Hardcoded per run?
3. **Secrets & Credentials Management**
    - How should credentials be handled?
        - Environment variables
        - Secure vault (if any)
4. **Provisioning Validation**
    - Since stubs are used:
        - Do we validate via:
            - API responses?
            - DB checks (if accessible)?
            - UI confirmation only?

💡 This will define:

- Config management design
- Data factories / builders
- Reusability across products (Broadband/Mobile/VOIP)

Once you respond, I'll move to **Question 5**.

Ans4: test data is available in excel and can be used from that. Some of the scenarios have product specific data which are available as part of prod spec document. this needs to be integrated with the code. Tests will run in SIT and UAT and env switching is config based. Credentials should be handled using Environment variables. Through API responses and also DB checks are used for validation

Great — this gives us a solid foundation for **data-driven and environment-agnostic framework design**, especially with DB + API validation layered in.

Question 5 of 6: CI/CD, Execution & Reporting

Now let's define how the framework will **run in pipelines and report results**, which is critical for adoption.

☞ Please provide:

1. **CI/CD Tool**
    - Which tool will be used?
        - Jenkins
        - Azure DevOps
        - GitHub Actions
        - Others?
2. **Execution Strategy**
    - How should tests run?
        - Parallel execution (recommended)
        - Sequential for some flows (E2E dependencies?)
3. **Trigger Points**
    - When should automation run?
        - On every build (smoke?)
        - Nightly regression?
        - Pre-release full suite?
4. **Reporting Requirements**
    - Preferred reports:
        - Playwright HTML reports (default)
        - Allure reports (recommended for enterprise)
    - Do you need:
        - Screenshots on failure?
        - Video recording?
        - Logs (API + UI)?

💡 This will define:

- Pipeline design
- Reporting framework integration
- Debuggability and traceability

Once you answer, I'll ask the **final Question 6**, and then generate your **complete Automation Framework (architecture + folder structure + code design + best practices)**.

Ans5: Use jenkins as Ci/CD tool and tests run parallely and few would run sequential which has dependencies. Automation is run on every build and use Allure reports. Capture screenshots for failed test cases

Perfect — that's a solid enterprise-grade setup with Jenkins + Allure + parallel execution.

Question 6 of 6: Version Control, Coding Standards & Reusability

Final piece before I generate your full automation framework.

☞ Please provide:

1. **Version Control**
    - Which system will be used?
        - Git (GitHub / GitLab / Bitbucket)?
2. **Branching Strategy**
    - Any preference?
        - Feature branches + PRs (recommended)
        - Trunk-based?
3. **Coding Standards**
    - Do you want:
        - ESLint + Prettier (recommended for TypeScript)?
        - Any strict linting rules?
4. **Reusability & Utilities**
    - Should framework include:
        - Custom utilities (API client, DB client, logger)?
        - Common reusable components (login, order creation, provisioning validation)?

💡 This will finalize:

- Code governance
- Maintainability standards
- Framework extensibility

Once you answer, I'll generate a **complete industry-standard Automation Framework design**, including:

- Architecture diagram (explained)
- Folder structure
- Sample code structure
- Execution flow
- CI/CD integration
- Best practices tailored to telecom E2E testing

Ans6: Github is the version control and use maven for maintaining dependencies. Have the feature branches + PRs. Use recommended coding standards and maintain all reusable components separately.

Alright — I'm going to be direct: **Playwright + TypeScript + Maven** is not a natural combination. Maven is Java-centric, while your stack is Node-based. For a clean, maintainable framework, dependency management should be handled via **npm/yarn**.

☞ I'll proceed with **industry-standard Node-based dependency management**, while still aligning with your governance expectations (GitHub + PRs + structured builds).

Automation Framework Design

Telecom Fulfilment (Broadband, Mobile, VOIP)

---

1. Framework Overview

This framework is designed for:

- **End-to-End telecom fulfilment journeys**
- **Integrated API + UI automation**
- **Data-driven testing**
- **Scalable parallel execution via Jenkins**

Key Principles

- API-first validation layered with UI
- High reusability via modular design
- Clear separation of concerns
- Config-driven execution
- Maintainable and scalable

---

2. Architecture Design

Layered Architecture

```
Test Layer (Specs)
        ↓
Business Layer (Flows / Services)
        ↓
Page Object Layer (UI)
        ↓
API Client Layer
        ↓
Utility Layer (DB, Logger, Data)
        ↓
Core Config Layer
```

3. Tech Stack

Area	Tool
UI Automation	Playwright
Language	TypeScript
API Testing	Playwright API / Axios
Reporting	Allure
CI/CD	Jenkins
Version Control	GitHub
Linting	ESLint + Prettier

---

4. Folder Structure

```
/framework
  /config
    env.ts
    testConfig.ts

  /data
    testData.xlsx
    productData.json

  /utils
    excelReader.ts
    logger.ts
    dbClient.ts
    apiHelper.ts

  /api
    order.api.ts
    customer.api.ts
    provisioning.api.ts

  /pages
    login.page.ts
    order.page.ts
    product.page.ts

  /flows
    order.flow.ts
    provisioning.flow.ts
    customer.flow.ts

  /tests
    e2e/
      order.e2e.spec.ts
    api/
      order.api.spec.ts
```

```
/fixtures
  base.fixture.ts

/reports
```

---

5. Design Patterns Used

1. Page Object Model (POM)

- Encapsulates UI actions
- Improves maintainability

2. Service/Flow Layer

- Represents business workflows:
  - Order creation
  - Provisioning
  - Upgrade

3. Data-Driven Framework

- Excel-based test data
- Product spec integration

4. Factory Pattern (Test Data)

- Dynamic data creation for:
  - Customers
  - Orders

---

6. Test Data Management

Sources

- Excel (primary input)
- Product spec documents (JSON conversion recommended)

Implementation

```
const data = excelReader.getTestData("Broadband_Order");
```

Strategy

- Static + dynamic hybrid
- Unique data generation for E2E
- Reusable customer profiles

---

7. Environment Configuration

Config-Based Switching

```
export const config = {
  env: process.env.ENV || 'SIT',
  baseURL: process.env.BASE_URL
};
```

Supported Environments

- SIT
- UAT

---

8. Authentication Handling

- Centralized login utility
- Session reuse via Playwright storage state

```
await loginPage.login(username, password);
```

---

9. API Layer Design

Reusable API Client

```
class OrderAPI {
  async createOrder(payload) {
    return await apiHelper.post('/orders', payload);
  }
}
```

Usage in UI Flow

- Pre-condition setup
- Post-validation

---

10. Database Validation Layer

```
const result = await dbClient.query("SELECT * FROM orders WHERE id = ?");
expect(result.status).toBe("Provisioned");
```

---

11. Business Flow Layer

Example: Order Flow

```
class OrderFlow {
  async placeOrder(data) {
    await productPage.selectProduct(data.product);
```

```
    await orderPage.submitOrder();
  }
}
```

---

12. Test Implementation

E2E Test Example

```
test('Broadband Order E2E', async ({ page }) => {
  const data = getTestData('Broadband');

  await loginFlow.login();
  await orderFlow.placeOrder(data);

  const response = await orderAPI.getOrderStatus();
  expect(response.status).toBe('Provisioned');

  const dbData = await dbClient.getOrder(data.id);
  expect(dbData.status).toBe('Active');
});
```

---

13. Execution Strategy

Parallel Execution

- Default: Parallel
- Sequential: Tagged tests

```
npx playwright test --workers=5
```

Tagging Strategy

- @smoke
- @regression
- @e2e
- @sequential

---

14. CI/CD Integration (Jenkins)

Pipeline Stages

1. Checkout
2. Install dependencies
3. Run tests
4. Generate Allure report

Sample Pipeline

```
stage('Test') {
  steps {
```

```
    sh 'npx playwright test'
  }
}
```

---

15. Reporting (Allure)

Features

- Step-level reporting
- Screenshots on failure
- API logs

Integration

```
allure generate ./allure-results
```

---

16. Logging & Debugging

- Central logger utility
- API request/response logging
- Screenshot on failure

---

17. Coding Standards

- ESLint + Prettier enforced
- Strict typing (TypeScript)
- Modular reusable code
- Naming conventions:
    - Pages → *.page.ts
    - APIs → *.api.ts
    - Flows → *.flow.ts

---

18. Version Control Strategy

- GitHub
- Feature branches
- Pull Requests with review
- Mandatory code checks before merge

---

19. Reusability Components

- Login module
- Order creation module
- API client
- DB utilities

- Test data readers

---

20. Best Practices

- API-first validation
- Avoid flaky UI dependencies
- Use retries cautiously
- Keep tests independent
- Tag tests properly
- Maintain small reusable methods

---

21. Scalability Considerations

- Add microservice-level API tests
- Integrate contract testing
- Extend for mobile (future)
- Plug into cloud execution (e.g., browser grid)

---

22. Conclusion

This framework ensures:

- Robust E2E telecom validation
- High reusability and maintainability
- Faster execution via parallelism
- Strong validation using UI + API + DB layers

If you want next, I can:

- Generate a **ready-to-use GitHub repo structure (with actual code files)**
- Convert this into a **Word/PDF architecture document**
- Or create a **Jenkins pipeline + package.json fully wired setup**